Universidad Nacional Autónoma de México

Facultad de Ingeniería — División de Ingeniería Eléctrica

Diseño Arquitectónico y Guía de Implementación: Parser & SDT

Asignatura: Compiladores | **Grupo:** 5 | **Equipo:** 02

Paquete: `unam.fi.compilers.g5.02` | **Fecha límite:** 10 de noviembre de 2026

---

1 Justificación Arquitectónica y Enfoque Teórico

El presente documento define el diseño formal del Analizador Sintáctico (**Parser**) y del esquema de Traducción Dirigida por Sintaxis (**SDT**) para el compilador del Equipo 02. En apego a las especificaciones de evaluación de la asignatura, se adopta la construcción manual basada exclusivamente en los conceptos teóricos vistos en clase (lo cual otorga puntuación adicional sobre la evaluación total), descartando el uso de generadores de caja negra como PLY, SLY, YACC o Bison.

1.1 Selección del Método de Parsing: Predictivo Top-Down LL(1) con Pila Explícita

A partir de la taxonomía de analizadores sintácticos estudiada en las sesiones teóricas, se seleccionó un **Parser Predictivo Descendente LL(1) sin retroceso (*without backtracking*)**, implementado mediante una **Pila Explícita (*Stack*)**, un **Búfer de Entrada (*i/p Buffer*)** y una **Tabla de Análisis Sintáctico (*LL(1) Parsing Table*)**. Esta decisión se fundamenta en tres criterios técnicos:

1. **Trazabilidad Teórica para el Reporte y Defensa Oral:** A diferencia del método de descenso recursivo (que oculta el estado de derivación en la pila de llamadas interna de Python), la arquitectura con pila explícita permite imprimir y auditar en cada iteración la cuádrupla $(\text{Step}, \text{Stack}, \text{Input}, \text{Action})$. Esto proporciona la argumentación teórica exacta exigida para cada salida del reporte técnico.
2. **Ventaja frente a Analizadores Bottom-Up (LR(0), SLR(1), CLR(1), LALR(1)):** Los métodos ascendentes requieren construir la colección canónica de ítems LR mediante las operaciones `closure` y `goto`. Al incorporar el catálogo de palabras reservadas y tipos de datos de Solidity, el autómata de estados LR crece considerablemente y puede presentar conflictos *shift/reduce*. El modelo LL(1) opera directamente sobre los conjuntos FIRST() y FOLLOW() con complejidad temporal y espacial lineal.
3. **Desacoplamiento Estructural entre Sintaxis y Semántica (Procedimiento SDT):** Siguiendo el procedimiento de dos pasos establecido en los apuntes de *Syntax Directed Translation*:
    - **Paso 1:** El motor LL(1) valida la estructura gramatical de los tokens emitidos por el Lexer y construye el Árbol de Análisis Sintáctico (*Parse Tree*). Si la pila alcanza el símbolo de fin de entrada `$` al mismo tiempo que el búfer, se garantiza la validez sintáctica (`Parsing Success!`).
    - **Paso 2:** Se realiza un recorrido (*traversal*) sobre el árbol construido ejecutando las acciones semánticas asociadas a cada regla de producción para validar tipos y declaraciones en la Tabla de Símbolos, logrando separar limpiamente un éxito sintáctico de una falla semántica (`Parsing Success!` seguido de `SDT error...`).

2 Arquitectura del Paquete (`unam.fi.compilers.g5.02`)

Siguiendo buenas prácticas de Ingeniería de Software, el paquete existente se extiende incorporando los módulos de análisis sintáctico, árbol sintáctico y verificador SDT, manteniendo el desacoplamiento por capas:

```
unam/
\-- fi/
    \-- compilers/
        \-- g5/
            \-- 02/
                |-- __init__.py
                |-- main.py              # Orquestador: conecta Lexer -> Parser ->
    ↪  SDT
                |-- io/
                |   |-- reader.py        # Carga de archivo o cadena fuente en
    ↪ memoria
                |   \-- writer.py        # Salida estandarizada y reporte de linea
    ↪  de error
                |-- lexer/
                |   |-- scanner.py       # Generador del flujo de tokens y Panic-
    ↪ Mode
                |   \-- dfa.py           # Automata Finito Determinista (State
    ↪ Pattern)
                |-- parser/
                |   |-- ll1_parser.py    # Motor LL(1): Pila explicita, Buffer y
    ↪ Tabla
                |   \-- grammar.py       # Producciones CFG, conjuntos FIRST,
    ↪ FOLLOW y Matriz
                |-- sdt/
                |   \-- analyzer.py      # Recorrido del Parse Tree y reglas
    ↪ semanticas SDT
                |-- symbol_table/
                |   \-- symtab.py        # Operaciones Insert(), Lookup(), Set() y
    ↪  Reset()
                \-- models/
                    |-- token.py         # Objeto Token <Category, Lexeme, Line>
                    \-- ast_node.py      # Nodo del Parse Tree con atributos
    ↪ sintetizados
```

3 Diseño de la Gramática Libre de Contexto (CFG) y Tabla LL(1)

El analizador sintáctico recibe el flujo de tokens generado por el analizador léxico dentro de las siete categorías del proyecto (`Keyword`, `Identifier`, `Operator`, `Constants`, `Literals`, `Punctuators` y `Special_Characters`). Para distinguir palabras reservadas de control (`if`, `else`, `while`, `for`, `printf`) y tipos de datos (`int`, `real`, `string`, `bool`), el motor LL(1) evalúa el lexema específico de los tokens de categoría `Keyword`, `Operator` y `Punctuators`, tal como se modela en los apuntes de clase.

3.1 Punto y Coma Obligatorio, Estructuras de Control y Precedencia

En apego a los apuntes teóricos ($S \rightarrow \text{id} = E;$), toda sentencia simple (declaración, asignación e impresión con `printf`) exige obligatoriamente el delimitador punto y coma (`;`) al final. Asimismo, para incorporar estructuras de control (`if-else`, `while`, `for`) sin introducir la ambigüedad clásica del *dangling-else* en una gramática LL(1), los cuerpos de bloque se delimitan obligatoriamente entre llaves `{ StmtList }`.

Aplicando eliminación de recursividad por la izquierda ($A \to \beta A'$, $A' \to \alpha A' \mid \epsilon$) y factorización por la izquierda (*Left Factoring*), se obtiene la siguiente CFG determinista de 24 reglas:

A. Estructura de Programa y Sentencias (*Statements*):

1. Program $\to$ StmtList
2. StmtList $\to$ Stmt StmtList $\mid \epsilon$
3. Stmt $\to$ DeclStmt $\mid$ AssignStmt $\mid$ PrintStmt $\mid$ IfStmt $\mid$ WhileStmt $\mid$ ForStmt
4. DeclStmt $\to$ Type Identifier DeclRest ;
5. DeclRest $\to = E \mid \epsilon$
6. AssignStmt $\to$ Identifier $= E$;
7. PrintStmt $\to$ `printf` (E ArgList) ;
8. ArgList $\to$, E ArgList $\mid \epsilon$
9. IfStmt $\to$ `if` (E) { StmtList } ElsePart
10. ElsePart $\to$ `else` { StmtList } $\mid \epsilon$
11. WhileStmt $\to$ `while` (E) { StmtList }
12. ForStmt $\to$ `for` (ForInit ; E ; ForUpdate) { StmtList }
13. ForInit $\to$ Type Identifier $= E \mid$ Identifier $= E$
14. ForUpdate $\to$ Identifier $= E$
15. Type $\to$ `int` $\mid$ `real` $\mid$ `string` $\mid$ `bool`

B. Jerarquía de Expresiones sin Recursividad Izquierda:

16. $E \to R\ E'$ *(Nivel Lógico: &&, ||)*
17. $E' \to$ OpLog $R\ E' \mid \epsilon$ donde OpLog $\in \{\&\&, ||\}$
18. $R \to A\ R'$ *(Nivel Relacional e Igualdad: <, >, <=, >=, ==, !=)*
19. $R' \to$ OpRel $A\ R' \mid \epsilon$ donde OpRel $\in \{<, >, <=, >=, ==, !=\}$
20. $A \to T\ A'$ *(Nivel Aditivo: +, -)*
21. $A' \to$ OpAdit $T\ A' \mid \epsilon$ donde OpAdit $\in \{+, -\}$
22. $T \to F\ T'$ *(Nivel Multiplicativo: *, /)*
23. $T' \to$ OpMult $F\ T' \mid \epsilon$ donde OpMult $\in \{*, /\}$
24. $F \to$ (E) $\mid$! $F \mid$ Identifier $\mid$ Constants $\mid$ Literals

3.2 Conjuntos FIRST() y FOLLOW()

Aplicando las tres reglas teóricas vistas en clase para el cálculo de conjuntos de primer símbolo terminal (FIRST) y símbolo siguiente (FOLLOW), se obtiene la Tabla 1.

No-Terminal	Conjunto FIRST	Conjunto FOLLOW
Program	{ `int`, `real`, `string`, `bool`, id, `printf`, `if`, `while`, `for`, ϵ }	{ $ }
StmtList	{ `int`, `real`, `string`, `bool`, id, `printf`, `if`, `while`, `for`, ϵ }	{ }, $ }
Stmt	{ `int`, `real`, `string`, `bool`, id, `printf`, `if`, `while`, `for` }	{ FIRST(Stmt), }, $ }
DeclStmt	{ `int`, `real`, `string`, `bool` }	{ FIRST(Stmt), }, $ }
DeclRest	{ =, ϵ }	{ ; }
AssignStmt	{ Identifier }	{ FIRST(Stmt), }, $ }
PrintStmt	{ `printf` }	{ FIRST(Stmt), }, $ }
ArgList	{ ,, ϵ }	{) }
IfStmt	{ `if` }	{ FIRST(Stmt), }, $ }
ElsePart	{ `else`, ϵ }	{ FIRST(Stmt), }, $ }
WhileStmt/ForStmt	{ `while` } / { `for` }	{ FIRST(Stmt), }, $ }
ForInit	{ `int`, `real`, `string`, `bool`, Identifier }	{ ; }
ForUpdate	{ Identifier }	{) }
E	{ (, !, Identifier, Constants, Literals }	{ ;,), , }
E'	{ &&, ||, ϵ }	{ ;,), , }
R	{ (, !, Identifier, Constants, Literals }	{ &&, ||, ;,), , }
R'	{ <, >, <=, >=, ==, !=, ϵ }	{ &&, ||, ;,), , }
A	{ (, !, Identifier, Constants, Literals }	{ OpRel, OpLog, ;,), , }
A'	{ +, -, ϵ }	{ OpRel, OpLog, ;,), , }
T	{ (, !, Identifier, Constants, Literals }	{ +, -, OpRel, OpLog, ;,), , }
T'	{ *, /, ϵ }	{ +, -, OpRel, OpLog, ;,), , }
F	{ (, !, Identifier, Constants, Literals }	{ *, /, +, -, OpRel, OpLog, ;,), , }

Cuadro 1: Conjuntos FIRST y FOLLOW para la gramática extendida con estructuras de control.

3.3 Construcción de la Tabla de Parsing LL(1)

Aplicando las reglas de construcción de la tabla LL(1) vistas en clase (producciones regulares bajo su conjunto FIRST y producciones ϵ bajo su conjunto FOLLOW), ninguna celda presenta colisiones, demostrando que la gramática es estrictamente LL(1).

Tipo	Aritméticos (`+`, `-`, `*`, `/`)	Relacionales (`<`, `>`, `<=`, `>=`)	Igualdad (`==`, `!=`)	Lógicos (`&&`, `||`, `!`)	Asignación (`=`)
`int`	`int` OP `int` → `int`	`int` OP `int` → `bool`	`int` OP `int` → `bool`	Prohibido (`error`)	Recibe solo `int`
`real`	`real` OP `real` → `real`	`real` OP `real` → `bool`	`real` OP `real` → `bool`	Prohibido (`error`)	Recibe solo `real`
`string`	Solo `string` + `string` → `string`	Prohibido (`error`)	`string` OP `string` → `bool`	Prohibido (`error`)	Recibe solo `string`
`bool`	Prohibido (`error`)	Prohibido (`error`)	`bool` OP `bool` → `bool`	`bool` OP `bool` → `bool`	Recibe solo `bool`

Cuadro 2: Matriz de compatibilidad semántica de tipos para el verificador SDT.

4 Esquema SDT, Matriz de Tipos y Tabla de Símbolos

En cumplimiento con el modelo teórico CFG+Semantic Rules $\Rightarrow$ SDT, la verificación semántica se ejecuta recorriendo el *Parse Tree* en posorden (*bottom-up traversal*) una vez que el Parser LL(1) ha validado la sintaxis.

4.1 Matriz de Operaciones Permitidas por Tipo de Dato

Se establece un sistema de **tipado estricto** sobre cuatro tipos primitivos: `int`, `real`, `string` y `bool`. Cualquier operación fuera de las celdas válidas de la Tabla 3 sintetiza el atributo `error_type` y dispara SDT `error...` indicando la línea exacta.

Tipo	**Aritméticos (+, -, *, /)**	**Relacionales (<, >, <=, >=)**	**Igualdad (==, !=)**	**Lógicos (&&, ||, !)**	**Asignación (=)**
int	`int` OP `int` → `int`	`int` OP `int` → `bool`	`int` OP `int` → `bool`	Prohibido (`error`)	Recibe solo `int`
real	`real` OP `real` → `real`	`real` OP `real` → `bool`	`real` OP `real` → `bool`	Prohibido (`error`)	Recibe solo `real`
string	Solo `string` + `string` → `string`	Prohibido (`error`)	`string` OP `string` → `bool`	Prohibido (`error`)	Recibe solo `string`
bool	Prohibido (`error`)	Prohibido (`error`)	`bool` OP `bool` → `bool`	`bool` OP `bool` → `bool`	Recibe solo `bool`

Cuadro 3: Matriz de compatibilidad semántica de tipos para el verificador SDT.

4.2 Reglas Semánticas SDT para Sentencias y Expresiones

Producción Gramatical (CFG)	**Regla Semántica Asociada (Acción SDT)**
DeclStmt $\rightarrow$ Type id DeclRest ;	`{ entry = Lookup(id.lexeme); if entry.type != "TBD": error("Variable redeclarada", id.line); if DeclRest.type != "void" and DeclRest.type != Type.type: error("Tipo incompatible"); else: Set(id.lexeme, Type=Type.type, Size=sizeof(Type.type), Dimension=0) }`
AssignStmt $\rightarrow$ id $= E$;	`{ entry = Lookup(id.lexeme); if entry.type == "TBD": error("Variable no declarada", id.line); elif entry.type != E.type: error("Incompatibilidad en asignación", id.line); else: Set(id.lexeme, LineOfUsage=id.line) }`
PrintStmt $\rightarrow$ `printf` (E ArgList) ;	`{ if E.type == "error_type" or ArgList.type == "error_type": error("Argumento inválido en printf", printf.line) }`
IfStmt $\rightarrow$ `if` (E) {StmtList} ElsePart	`{ if E.type != "bool": error("La condición de if debe ser bool", if.line); verify(StmtList); verify(ElsePart) }`
WhileStmt $\rightarrow$ `while` (E) {StmtList}	`{ if E.type != "bool": error("La condición de while debe ser bool", while.line); verify(StmtList) }`
ForStmt $\rightarrow$ `for` (ForInit; E; ForUpdate) $\{\dots\}$	`{ verify(ForInit); if E.type != "bool": error("La condición de for debe ser bool", for.line); verify(ForUpdate); verify(StmtList) }`
$E \rightarrow R\,E'$ / $R \rightarrow A\,R'$	`{ Evalúa compatibilidad lógica (bool) y relacional (→ bool) según Tabla 3 }`
$A \rightarrow T\,A'$ / $T \rightarrow F\,T'$	`{ Evalúa compatibilidad aditiva y multiplicativa según Tabla 3 }`
$F \rightarrow$ Constants | Literals | Identifier	`{ Constants: "int" o "real" (si contiene '.'); Literals: "bool" o "string"; Identifier: Lookup(id.lexeme).type (error si es "TBD") y actualiza LineOfUsage }`

Cuadro 4: Reglas del esquema SDT para sentencias simples, bloques de control y expresiones.

4.3 Atributos de cada Identificador y Funcionamiento de `Set()` y `Reset()`

De acuerdo con los apuntes de clase sobre la Tabla de Símbolos (*Symbol Table Entries*), cada identificador registrado mantiene 7 atributos estructurales, los cuales evolucionan entre la fase léxica (`Insert`) y la fase semántica (`Lookup`, `Set`, `Reset`):

1. **`Name`:** Lexema del identificador (ej. `$x`, `count`). Actúa como clave primaria.
2. **`Type`:** Inicializado en `"TBD"` por el Lexer. Cuando el SDT procesa un `DeclStmt` válido, invoca `Set()` para asignarle `"int"`, `"real"`, `"string"` o `"bool"`.
3. **`Size`:** Inicializado en `"TBD"`. El método `Set()` calcula y asigna automáticamente el tamaño en bytes según el tipo confirmado: 4 bytes para `int`, 8 bytes para `real`, 1 byte para `bool`, o la longitud de la cadena para `string`.

4. **Dimension:** Inicializado en `"TBD"`. Para variables escalares se actualiza a `0` mediante `Set()`.
5. **Line of declamation:** Línea donde el identificador fue declarado.
6. **Line of usage:** Lista dinámica de líneas donde la variable es referenciada en asignaciones, condiciones o llamadas a `printf`. Cada vez que el SDT evalúa un identificador en uso, invoca `Set(name, line_usage=line)` para anexar la línea actual.
7. **Address:** Inicializado en `"TBD"`. Cuando `Set()` confirma la declaración de una variable, un contador de memoria base (`0x1000`) le asigna su dirección hexadecimal consecutiva según su `Size` (ej. `0x1000`, `0x1004`).

Operación `Reset()` en bloques anidados: Al tratarse de un lenguaje estructurado por bloques (`if`, `while`, `for`), la Tabla de Símbolos soporta la operación `Reset()` para limpiar o restaurar las variables locales declaradas exclusivamente dentro de un bloque `{ ... }` al cerrar su llave `}`, o bien reiniciar la tabla entre ejecuciones independientes.

5 Trazas Teóricas y Argumentación de Salidas Esperadas

5.1 Caso 1: Éxito Sintáctico y Semántico (`int $x = 10;`)

Entrada: `int $x = 10;` (en línea 1).
Flujo de Tokens: `<Keyword, "int">`, `<Identifier, "$x">`, `<Operator, "=">`, `<Constants, "10">`, `<Punctuators, ";">`, `<Special_Characters, "$">`.

Step	Stack (Base → Tope)	Input Buffer (i/p)	Action (Tabla LL(1) / Match)
1	`$ Program`	`int $x = 10 ; $`	Program → StmtList
2	`$ StmtList`	`int $x = 10 ; $`	StmtList → Stmt StmtList
3	`$ StmtList Stmt`	`int $x = 10 ; $`	Stmt → DeclStmt
4	`$ StmtList ; DeclRest id Type`	`int $x = 10 ; $`	Type → `int`, luego `match("int")`
5	`$ StmtList ; DeclRest id`	`$x = 10 ; $`	`match Identifier ("$x")`
6	`$ StmtList ; DeclRest`	`= 10 ; $`	DeclRest → $= E$, luego `match("=")`
7	`$ StmtList ; E`	`10 ; $`	Deriva $E \Rightarrow R \Rightarrow A \Rightarrow T \Rightarrow F \rightarrow$ Constants
8	`$ StmtList ; E' R' A' T' Constants`	`10 ; $`	`match Constants ("10")`
9	`$ StmtList ; E' R' A' T'`	`; $`	Reducciones ϵ en T', A', R', E' ante `;`
10	`$ StmtList ;`	`; $`	`match Punctuators (";")`
11	`$ StmtList`	`$`	StmtList → ϵ
12	`$`	`$`	**`Parsing Success!`**

Cuadro 5: Traza en pila del Parser LL(1) con punto y coma obligatorio.

Evaluación SDT: La constante `10` sintetiza `int` hasta el nodo E. En `DeclStmt`, `Lookup("$x")` confirma que su tipo es `"TBD"` y que `Type ("int") == DeclRest ("int")`. Se ejecuta `Set("$x", Type="int", Size=4, Dimension=0, Address="0x1000")` y el sistema imprime:

```
Parsing Success!
SDT Verified!
```

5.2 Caso 2: Éxito Sintáctico pero Error Semántico (SDT `error...`)

Ejemplos de Entrada (en línea 1):

- **Incompatibilidad en declaración:** `int $x = "diez";`
- **Condición no booleana en control de flujo:** `if (10 + 20) { $x = 1; }`

Argumento Teórico: Ambos fragmentos cumplen al 100% las reglas sintácticas de la CFG (en el primero, `"diez"` es un literal válido para F; en el segundo, `10 + 20` es una expresión válida E entre paréntesis seguida de un bloque con llaves), por lo que el Parser LL(1) vacía la pila con éxito e imprime `Parsing Success!`. Sin embargo, durante el recorrido posorden del SDT, el primer caso falla porque `int` $\neq$ `string`, y el segundo falla porque la regla semántica de `IfStmt` exige que E.type $==$ `"bool"`, pero `10 + 20` sintetiza `"int"`:

```
Parsing Success!
SDT error...
error in line 1
```

5.3 Caso 3: Error Sintáctico y Error Semántico (`Parsing error...`)

Ejemplos de Entrada (en línea 1): `int $x = 10` (omisión de `;`) o `int $x = * 10;` (operador mal ubicado).
Argumento Teórico: Al omitir el punto y coma obligatorio, cuando el tope de la pila espera hacer *match* con el terminal `;` pero el *lookahead* es el fin de entrada `$`, ocurre una discrepancia terminal (`;` $\neq$ `$`). De igual forma, ante `* 10`, la celda $M[E, *]$ de la tabla LL(1) está vacía. Se aborta el árbol sintáctico y se reporta el fallo en la línea correspondiente:

```
Parsing error...
SDT error...
error in line 1
```